Documentação das Otimizações - Ajuste Polinomial

Resumo Executivo

Este documento detalha as otimizações implementadas no código de ajuste polinomial, transformando o algoritmo original em uma versão significativamente mais eficiente. As otimizações abordam aspectos fundamentais de performance: redução de operações matemáticas, melhoria da localidade de cache, paralelização implícita e estruturas de dados mais eficientes.

Análise do Código Original

Problemas Identificados:

1. **Recálculo desnecessário de potências**: `pow(x[k], i+j)` calculada repetidamente
2. **Má localidade de cache**: Acessos não sequenciais à memória
3. **Operações redundantes**: Matriz simétrica calculada duas vezes
4. **Avaliação polinomial ineficiente**: Uso direto de `pow()` múltiplas vezes
5. **Falta de loop unrolling**: Perdas de performance em loops pequenos

Complexidade Original:

- Montagem do sistema: $O(n^2 \times p \times \log(\text{max_power}))$ devido ao uso de `pow()`
- Eliminação de Gauss: $O(n^3)$
- Avaliação de resíduos: $O(p \times n \times \log(\text{max_power}))$

Otimizações Implementadas

1. Pré-computação de Potências (Otimização Fundamental)

Antes:

c

```c
A[i][j] += pow(x[k], i+j);  // O(log(i+j)) por operação
```

Depois:

c

```c
// Pré-computa todas as potências necessárias
for (int pow = 1; pow <= maxPower; ++pow) {
    xPowers[i][pow] = xPowers[i][pow-1] * x[i];  // O(1) por operação
}
```

Benefícios:

- Reduz complexidade de $O(p \times n^2 \times \log(max_power))$ para $O(p \times max_power + n^2)$
- Elimina chamadas custosas para `pow()`
- Melhora significativamente a performance para graus altos

2. Exploração da Simetria da Matriz

Implementação:

```c
for (int j = i; j < n; ++j) {  // Apenas metade superior
    double term = xPowers[k][i + j];
    A[i][j] += term;
    if (i != j) {
        A[j][i] += term;  // Copia para metade inferior
    }
}
```

Benefícios:

- Reduz cálculos da matriz de n^2 para $n(n+1)/2$
- Melhora localidade de cache ao acessar elementos próximos
- Economia de ~50% das operações na montagem da matriz

3. Loop Unrolling Manual

Implementação na Eliminação de Gauss:

```c
int j = i + 1;
for (; j < n - 3; j += 4) {  // Processa 4 elementos por iteração
    A[k][j] -= A[i][j] * m;
    A[k][j+1] -= A[i][j+1] * m;
    A[k][j+2] -= A[i][j+2] * m;
    A[k][j+3] -= A[i][j+3] * m;
}
```

Benefícios:

- Reduz overhead de controle de loop
- Melhora pipeline do processador
- Permite melhor uso de registradores

- Ganho típico de 15-30% em loops pequenos

4. Método de Horner para Avaliação Polinomial

Antes:

```c
double Px = alpha[0];
for (int i = 1; i <= N; ++i)
    Px += alpha[i]*pow(x,i);  // O(N × log(N))
```

Depois:

```c
double result = alpha[N];
for (int i = N - 1; i >= 0; --i) {
    result = result * x + alpha[i];  // O(N)
}
```

Benefícios:

- Reduz complexidade de O(N × log(N)) para O(N)
- Elimina todas as chamadas para `pow()`
- Melhora estabilidade numérica
- Mais cache-friendly (acesso sequencial aos coeficientes)

5. Otimização de Memória e Cache

Alocação Contígua:

```c
double *A_data = aligned_alloc(32, sizeof(double) * n * n);
for (int i = 0; i < n; ++i) {
    A[i] = A_data + i * n;  // Memória contígua
}
```

Benefícios:

- Melhora localidade espacial do cache
- Reduz cache misses
- Alinhamento de 32 bytes otimiza para instruções SIMD
- Acesso sequencial mais eficiente

6. Pivoteamento Otimizado

Implementação:

```c
int iMax = i;
double maxVal = fabs(A[i][i]);
for (int k = i + 1; k < n; ++k) {
    double absVal = fabs(A[k][i]);
    if (absVal > maxVal) {
        maxVal = absVal;
        iMax = k;
    }
}
```

Benefícios:

- Evita recálculo de `fabs()` desnecessário
- Melhor estabilidade numérica
- Reduz trocas de linhas desnecessárias

7. Processamento Vetorizado de Resíduos

Implementação:

```c
void calcularResiduos(double *x, double *y, long long int p, int N, double *alpha, dou
    for (long long int i = 0; i < p; ++i) {
        residuos[i] = fabs(y[i] - P_horner(x[i], N, alpha));
    }
}
```

Benefícios:

- Separação clara de responsabilidades
- Melhor reutilização de cache entre x, y e residuos
- Facilita futuras otimizações SIMD

Análise de Performance

Complexidade Temporal:

Original:

- Montagem: $O(n^2 \times p \times \log(2n))$
- Resolução: $O(n^3)$
- Resíduos: $O(p \times n \times \log(n))$
- **Total: $O(n^2 \times p \times \log(n) + n^3 + p \times n \times \log(n))$**

Otimizada:

- Pré-computação: $O(p \times 2n)$
- Montagem: $O(n^2 \times p)$
- Resolução: $O(n^3)$ [com melhor constante]
- Resíduos: $O(p \times n)$
- **Total: $O(n^2 \times p + n^3 + p \times n)$**

Ganhos Esperados:

1. **Para p >> n (muitos pontos):** Ganho de $\sim\log(2n) \approx$ 3-5x para n = 10-20
2. **Para n pequeno (< 20):** Loop unrolling adiciona 15-30%
3. **Cache optimization:** 20-40% dependendo da arquitetura
4. **Método de Horner:** 2-4x na avaliação de resíduos

Casos de Uso Otimais:

- **Melhor caso:** p >> n, n ≤ 20, dados que cabem em cache L2
- **Ganho esperado total:** 5-15x para casos típicos
- **Memoria:** Uso adicional de $O(p \times 2n)$ para cache de potências

Considerações de Implementação

Compatibilidade:

- Usa apenas bibliotecas padrão C
- `aligned_alloc()` com fallback para `malloc()`
- Funciona em sistemas que não suportam alinhamento de memória

Estabilidade Numérica:

- Mantém pivoteamento parcial
- Método de Horner é numericamente mais estável
- Não introduz problemas de precisão

Escalabilidade:

- $O(p \times 2n)$ de memória adicional pode ser limitante para n muito grande

- Para n > 100, considerar técnicas diferentes (ex: decomposição QR)

Possíveis Extensões Futuras

1. **SIMD/Vetorização:** Usar instruções AVX/SSE para operações paralelas
2. **OpenMP:** Paralelizar loops principais
3. **Técnicas numéricas avançadas:** QR decomposition para melhor estabilidade
4. **Cache blocking:** Para matrizes muito grandes
5. **Pré-computação adaptativa:** Calcular apenas potências necessárias

Conclusão

As otimizações implementadas transformam um algoritmo $O(n^2 \times p \times \log(n))$ em $O(n^2 \times p)$, com constantes significativamente menores. A combinação de técnicas algorítmicas (pré-computação, Horner) e de sistema (cache optimization, loop unrolling) resulta em ganhos substanciais de performance, especialmente para os casos de uso típicos do ajuste polinomial.

O código mantém a mesma interface e precisão numérica, tornando-se um drop-in replacement eficiente para a versão original.